

محتاجك تفكر معايا كمحترف في موضوع الـ STT Speech to Text. عايز أعمل تطبيق يشتغل على ويندوز 11 إن أنا أتكلم باللغة اللهجة المصرية أو اللغة العربية واللهجة المصرية طبعًا، وهو يترجم الكلام ده لحظي أو يترجم الكلام ده على طول إلى نص. وبإيت طبعًا يكون النص ده يتم دمجها أو تصطيب البرنامج ده فيبخرش على هيئة plugins داخل الورد الخاص 365، بحيث إن أقدر أقف بالماوس داخل أي ملف وورد وأتكلم ويكتبه على طول مباشرة، زي فكرة الـ dictation اللي موجودة في الورد، ولكن الميزة اللي هنا إنه هيكو completely offline يعني مش محتاج إنترنت. فاعمل search كده على أفضل models وأخف models اللي ممكن تشتغل باللغة العربية واللهجة المصرية، وفي نفس الوقت تكون سريع وقوي، مش محتاج إنترنت نهائي، وفي نفس الوقت خفيف على الجهاز، واعطيني فكرة كويسة.

هتعامل معاها كمنتج فعلي وليس مجرد تجربة STT: **محرك عربي/مصري يعمل Offline بالكامل + إدخال مباشر داخل Word 365 + استجابة شبه لحظية.** سأقارن بين المحركات من ناحية الدقة مع اللهجة المصرية، السرعة على CPU، استهلاك الذاكرة، وسهولة دمجها داخل ويندوز.

ظهر لي نقطتان مهمتان أثناء البحث: يوجد الآن نماذج متخصصة فعلاً في اللهجة المصرية، وليس مطلوبًا الاعتماد على Whisper العام فقط. وفي المقابل، أفضل تكامل مع Word عندك ليس Office Web Add-in؛ لأن نموذج Office.js يعتمد على مكوّن ويب/استضافة، بينما VSTO/COM Add-in على Word Desktop أنسب بكثير لتطبيق Offline حقيقي.

أيوه، والفكرة قابلة للتنفيذ جدًا على Windows 11 وبـ Offline كامل. والمطلوب هنا تحديدًا هو Speech-to-Text / Transcription وليس Translation: تتكلم مصري/عربي، والنص يظهر عند مؤشر الكتابة في Word مثل Dictation، لكن بدون إرسال الصوت أو النص لأي Cloud.

بعد البحث في النماذج الحالية، أنا لا أنصح أن نبدأ بـ Whisper Large أو بخدمة سحابية. أرى أن أفضل تصميم عندك هو **محرك محلي مستقل + Word VSTO Add-in**، مع اختيار نموذج مصري خفيف.

النماذج التي تستحق التجربة فعلاً

النموذج	مصري	الحجم/الثقل	CPU	Real-time	رأيي
Faster-Whisper Small Egyptian	ممتاز	MB 489~ model	ممتاز مع INT8	ممتاز	★ أفضل نقطة بداية
EgyptTalk-ASR-v2	ممتاز جدًا	MB 459~ model	يحتاج اختبار	Streaming capable	★ منافس قوي جدًا
fabhaus Whisper-	مصري	244M ~ params	جيد بعد CT2	جيد	واعد جدًا، يحتاج Benchmar

النموذج	مصري	الحجم/الثقل	CPU	Real-time	رأيي
small-eg2					k
Whisper Small الأصلي	عربي عام	244M	جيد	جيد	Baseline
Whisper Turbo	عربي عام	809M	أثقل	سريع مع GPU	أكبر من المطلوب
Vosk Arabic	عربي عام	MB 318	خفيف جدًا	ممتاز	للأجهزة الضعيفة
Nawah-ASR-118M-v5	عربي	118M	سريع جدًا	ممتاز	لا أفضله للمصري حاليًا

السبب في أنني أميل جدًا إلى **Faster-Whisper Small Egyptian** هو وجود نسخة مصرية محوَّلة بالفعل إلى CTranslate2 بحجم حوالي **MB 489** ومصممة للعمل مباشرة مع faster-whisper؛ ويمكن تشغيلها على CPU باستخدام (Hugging Face) INT8.

وهذا مهم جدًا لأن Faster-Whisper نفسه أسرع وأخف من تشغيل PyTorch Whisper التقليدي. في Benchmark المنشور للمشروع، نموذج Whisper Small على Intel i7-12700K عالج 13 دقيقة صوت في حوالي **1 دقيقة و42 ثانية باستخدام CPU INT8** واستهلك قرابة **1.48 GB RAM**؛ أي أسرع بكثير من الزمن الحقيقي على هذا المعالج. (GitHub)

لكن EgypTalk-ASR-v2 يستحق اهتمامًا خاصًا

هذا نموذج مصري حقيقي مبني على **NVIDIA FastConformer Hybrid Large**، وتم تدريبه على أكثر من **200 ساعة من تسجيلات اللهجة المصرية** تشمل الكلام العفوي، المقابلات، الإعلام، والكلام المقروء. والأهم أن معماريته موصوفة بأنها **low-latency و-streaming-capable**. حجم ملف النموذج المنشور حوالي 459 MB. (Hugging Face)

وهذا يجعله ربما أفضل من Whisper من الناحية المعمارية لتطبيق:

Microphone → كلام → نص فوري

لأن Whisper في الأصل ليس Streaming ASR حقيقيًا؛ نحن نجعله شبه لحظي باستخدام sliding/chunked inference، بينما FastConformer مصمم بصورة أفضل لهذا النوع من التشغيل.

لكن هناك عيب عملي: EgypTalk يعتمد على **NVIDIA NeMo + PyTorch**، وبالتالي توزيعه كبرنامج Windows صغير ونظيف أصعب قليلًا من Faster-Whisper/CTranslate2.

لذلك عند بناء المنتج سأختبر الاثنين، وليس واحدًا فقط.

هناك نموذج مصري آخر لفت انتباهي جدًا

fabhaus/whisper-small-eg2 مبني على Whisper Small، وينشر مطوره $WER = 8.33\%$ على مجموعة تقييم Egyptian Arabic الخاصة به. ([Hugging Face](#))

الرقم ممتاز جدًا، لكن يجب التعامل معه بحذر؛ لأنه **self-reported** وعلى Dataset محددة بحوالي 6 آلاف عينة، ولذلك لا يصح أن نقول إنه أفضل نموذج مصري بناءً على هذا الرقم وحده.

أنا سأحوّله إلى CTranslate2 INT8 وأضعه في الاختبار أمام النموذج السابق.

وهذه نقطة مهمة جدًا في مشروعك: لا نقارن WER المنشور بين النماذج مباشرة إذا كانت ال datasets مختلفة.

التصميم الذي أقترحه للتطبيق

أراه بهذا الشكل:

Microphone



Windows WASAPI / NAudio



16 kHz Mono PCM



Voice Activity Detection



Egyptian ASR Engine



Arabic Text Normalizer



Voice Commands Processor



Word VSTO Add-in



Word Cursor position داخل

وهنا يوجد قرار معماري مهم جدًا: لا تجعل Word نفسه يشغل نموذج الذكاء الاصطناعي.

اجعل هناك برنامجًا صغيرًا يعمل في الخلفية مثل:

ArabicDictationEngine.exe

Word Plugin يتواصل معه محليًا فقط.

مثلاً:

Word Add-in ⇌ Named Pipe ⇌ STT Engine

ولا أفضل localhost HTTP أصلاً إذا كان الهدف منتجًا Offline مغلقًا؛ Named Pipes أبسط وأكثر إحكامًا محليًا.

كيف سيكون الاستخدام؟

تفتح Word 365.

تجد Ribbon جديدًا مثل:

Offline Arabic Dictation

وفيه:

Start Dictation 

Stop 

Egyptian Arabic

Modern Standard Arabic

Auto Punctuation

Voice Commands

Custom Dictionary

وتضع المؤشر مثلاً هنا:

تبين من فحص المستند محل الفحص ...
وتبدأ الكلام:

“تبين من فحص المستند محل الفحص وجود اختلاف في الخصائص الخطية للتوقيع نقطة سطر جديد كما تبين...”

فيكتب:

تبين من فحص المستند محل الفحص وجود اختلاف في الخصائص الخطية للتوقيع.
كما تبين...

كل هذا بدون إنترنت.

Microsoft نفسها توثق أن VSTO Add-ins تستطيع التعامل مباشرة مع الـ Range و Selection في Word وإدخال النص في موضع المؤشر. ([Microsoft Learn](#))

وهذا بالضبط ما نحتاجه.

لماذا VSTO وليس Office.js؟

Office.js ممتاز للـ cross-platform، لكن بنيته في الأساس Web Add-in وتستخدم web application / web hosting. ([Microsoft Learn](#))

أما مشروعك فهو محدد جدًا:

Windows 11 + Word 365 Desktop + Offline

وهذا يجعل:

C# VSTO / COM Add-in

هو الاختيار المنطقي.

وسيكون عندك تكامل أقوى مع Word والـ cursor والـ document والـ Ribbon.

الجزء الأهم: كيف نجعل الكتابة “لحظية”؟

لو أخذنا الصوت كل 10 أو 20 ثانية ثم نسخناه، سيبدو البرنامج سيئًا جدًا.

الأصح أن يكون لدينا Ring Buffer، مثلًا:

Audio continuously captured

ثم كل حوالي:

ms 800-500

نراجع الصوت.

مع window أكبر مثلاً:

seconds 8-4

Overlap حوالي:

seconds 1.5-1

ثم نقسم النص إلى:

Partial text

و

Committed text

مثلاً تقول:

“تبين من فحص التوقيع...”

قد يعرض مؤقتاً:

تبين من فحص التوقيع...

ثم عندما يتأكد:

تبين من فحص التوقيع

نرسله إلى Word.

هذه الطريقة تمنع مشكلة أن Whisper يكتب كلمة ثم يرجع يغيرها.

و faster-whisper يحتوي أصلاً على **Silero Voice Activity Detection** ويمكن ضبط مدة الصمت المستخدمة لتحديد نهاية الجملة. ([GitHub](#))

ميزة مهمة جداً أضيفها للبرنامج

أنا لن أجعله STT فقط.

سأضع Voice Command Layer عربية.

مثلاً تقول:

“نقطة”

→ .

“فاصلة”

→ ,

“علامة استفهام”

→ ؟

“سطر جديد”

Enter →

“فقرة جديدة”

Paragraph →

“امسح آخر كلمة”

Delete last word →

“امسح آخر جملة”

Delete sentence →

“تراجع”

Undo →

“اكتب عنوان”

Heading mode →

وهذا سيجعل التطبيق أقرب فعلاً إلى Microsoft Dictation وليس مجرد نموذج ASR.

والأقوى بالنسبة لاستخدامك

يمكننا إنشاء Custom Vocabulary Offline.

مثلاً البرنامج يتعلم/يفضّل كلمات مثل:

التزييف

التزوير

مضاهاة

التوقيعات

المحرر

الفحص الفني

البصمة

Inkjet

Laser Printer

Word Error Rate

ROC

AUC

والأسماء الشخصية والمؤسسات.

ويمكن عمل Profiles:

General Arabic

Egyptian

Legal

Forensic

Banking

وهذا سيعطيك فارقًا كبيرًا عن البرامج العامة.

بالنسبة للأجهزة الضعيفة

هنا نستطيع عمل **3 مستويات للنموذج** داخل البرنامج:

Eco Mode

Vosk / model خفيف جدًا.

Balanced Mode — Default

Egyptian Whisper Small INT8.

Accuracy Mode

EgypTalk أو نموذج أكبر إذا الجهاز يسمح.

لكن أعتقد أن **Balanced وحده قد يكون كافيًا جدًا**.

Vosk لديه نموذج عربي بحجم MB 318 وWER معلن 16.4% على MGB-2، وهو خفيف ويعمل Offline، لكنه ليس متخصصًا للمصري مثل الخيارات السابقة. ([Alpha Cephei](#))

نموذج لا أنصح به حاليًا رغم أنه جديد وخفيف

وجدت Nawah-ASR-118M-v5.

هو صغير جدًا: **118M parameters** ويذكر مطوره سرعة CPU حوالي **5.9x realtime**. لكن ال model card نفسه يوضح أن أدائه على conversational Egyptian يصل تقريبًا إلى WER 0.55، وأن قوته الأساسية حاليًا في (Hugging Face) Modern Standard Arabic. لذلك لا أختاره لمشروعك.

Whisper Large-v3 / Turbo ؟

Whisper الرسمي حاليًا تقريبًا:

Model	Parameters	Approx VRAM
Tiny	39M	GB 1~
Base	74M	GB 1~
Small	244M	GB 2~
Medium	769M	GB 5~
Turbo	809M	GB 6~
Large	1550M	GB 10~

(GitHub)

لذلك **Turbo** و **Large-v3 overkill** بالنسبة لتطبيق Dictation مصري على أجهزة مكتبية متنوعة.

النموذج المصري Small أكثر منطقية.

التركيبة التي أختارها الآن

لو سأبدأ البرمجة غدًا، سأبني Version 1 هكذا:

Frontend / Windows C# .NET + WPF

Word integration C# VSTO Add-in

Microphone NAudio + WASAPI

ASR Engine faster-whisper / CTranslate2

Primary Model Egyptian Whisper Small INT8

Second experimental engine EgypTalk-ASR-v2

VAD Silero VAD

Communication Windows Named Pipes

Arabic processing C#/Python local rules

Inno Setup أو **Installer** MSI

Internet غير مطلوب نهائيًا، حتى أول تشغيل، لأن ال model سيوضع داخل Installer.

وهناك ميزة إضافية سأضعها من البداية: البرنامج لا يعمل فقط داخل Word. يكون هناك **Global Dictation Mode** اختياري يستطيع الكتابة في Notepad و Outlook والمتصفح وأي TextBox باستخدام Windows UI Automation/keyboard injection، بينما Word يحصل على التكامل الأقوى من خلال VSTO.

والخطوة الاحترافية قبل اعتماد النموذج

لا أصدق أرقام Hugging Face وحدها. نعمل Dataset خاصة بنا من 50-100 تسجيل مصري تشمل الكلام الهادئ، السريع، اللهجة المصرية الواضحة، العربية الرسمية، مصطلحات قانونية/فنية، وضوءاء بسيطة. ثم نقيس لكل نموذج WER، CER، latency لأول كلمة، latency **لنهاية الجملة**، RAM، Real-Time Factor، CPU usage.

وأتوقع أن المنافسة النهائية ستكون بين **Faster-Whisper Egyptian Small** و **EgypTalk-ASR-v2**؛ الأول أسهل كثيرًا في التحويل إلى تطبيق Windows خفيف، والثاني معماريًا جذاب جدًا لل streaming المصري. ([Hugging Face](#))

إذا كنت ستنفذ المشروع فعليًا، فأنا أرى أن هذا يمكن أن يتحول إلى برنامج Windows احترافي صغير شبيه بـ Microsoft Dictation ولكن 100% Arabic/Egyptian + Offline، وليس مجرد Python script.